Part 1: Design Pattern Identification

For each scenario below, identify the **most appropriate design pattern** and **explain why** in 2-3 sentences.

Question 1

Scenario:
You are building a text editor that supports multiple formatting options (bold, italic, underline, font color). Users can apply any combination of these formats to selected text. The formatting behavior should be addable at runtime without modifying existing classes, and you want to avoid creating hundreds of subclasses like `BoldItalicText`, `BoldUnderlineText`, etc.

Which design pattern would you use?

Question 2

Scenario:
You are developing a database connection manager for a large enterprise application. Multiple parts of the application (user authentication, product catalog, order processing) all need to access the same database connection pool. Creating multiple connection pools would waste resources and potentially cause data inconsistency. Only one instance of the connection manager should exist throughout the application lifecycle. The manager should also provide global access point to retrieve this instance.

Which design pattern would you use?

Part 2: Apply Patterns

Scenario 1 – Apply Factory Method

Problem:
You are developing a logistics application. Initially, it only supports **Truck** transportation. Now you need to add **Ship** and **Airplane** transport. Each transport has a different `deliver()` method implementation. The client code should not be coupled to concrete transport classes.

Task:
Write Java Code implementing **Factory Method** pattern.

Scenario 2 – Apply Abstract Factory

Problem:
A furniture store sells products in different **furniture families**:

- **Victorian** (Chair, Sofa, CoffeeTable)
- **Modern** (Chair, Sofa, CoffeeTable)
- **ArtDeco** (Chair, Sofa, CoffeeTable)

Each product has methods like **sitOn()** (Chair), **lieOn()** (Sofa), **drinkOn()** (CoffeeTable). The client must be able to switch entire families at runtime.

Task:
Implement **Abstract Factory** pattern for this scenario.

Scenario 3 – Identify & Apply

A stock market system tracks the price of Apple shares. Several trading apps and portfolio dashboards need to update instantly whenever the price changes. Currently, adding a new app requires modifying the stock market's code. The market should automatically notify all registered apps about price changes without knowing what each app does.

Question: Which design pattern would you use to solve this problem?

Question: Write Code in Java.

Scenario 4 – Identify & Apply

A luxury car dealership lets customers customize their car online. The base model includes standard features. Customers can add a **sunroof**, **leather seats**, **premium sound system**, **heated steering wheel**, or **parking sensors**. Each addition increases the price and updates the feature list. The dealership receives thousands of custom combinations daily and cannot predefine every possible car configuration class.

Question: Which design pattern would you use to solve this problem?

Question: Also identify which Design principle will be used?

Question: Write Code in Java.

Scenario 5 –Fix the Code By Applying Singleton Pattern

A online ticket booking system for a movie theater has a **TicketCounter** class that issues seat numbers. Multiple users are booking tickets simultaneously. Currently, each user creates their own TicketCounter object, resulting in duplicate seat numbers being issued to different users. The theater needs exactly **one** TicketCounter instance to ensure every seat is booked only once.

Buggy Code (No Singleton Applied)

```
// Multiple TicketCounter instances cause duplicate seats

import java.util.*;

class TicketCounter {
   private int currentSeat = 1;
   private Set<Integer> bookedSeats = new HashSet<>();

   // Public constructor - anyone can create multiple instances
   public TicketCounter() {
      System.out.println(" New TicketCounter created!");
   }
```

```
    public int bookTicket(String userName) {
        int seatNumber = currentSeat++;
        bookedSeats.add(seatNumber);
        System.out.println(userName + " booked Seat #" + seatNumber);
        return seatNumber;
    }

    public int getTotalBookings() {
        return bookedSeats.size();
    }

    public void showBookedSeats() {
        System.out.println("Booked seats: " + bookedSeats);
    }
}

public class BuggyTicketSystem {
    public static void main(String[] args) {
        System.out.println("=== MOVIE TICKET BOOKING SYSTEM ===\n");

        // Each user creates their own counter!
        TicketCounter ahmed = new TicketCounter();
        ahmed.bookTicket("Ahmed");
        ahmed.bookTicket("Ahmed");

        TicketCounter hania = new TicketCounter();
        hania.bookTicket("Hania");

        TicketCounter asim = new TicketCounter();
        asim.bookTicket("Asim");
        asim.bookTicket("Asim");

        System.out.println("\n--- PROBLEM ---");
        System.out.println("Ahmed's counter bookings: " + ahmed.getTotalBookings());
        System.out.println("Hania's counter bookings: " + hania.getTotalBookings());
        System.out.println("Asim's counter bookings: " + asim.getTotalBookings());

        ahmed.showBookedSeats();
        hania.showBookedSeats();
        asim.showBookedSeats();

        System.out.println(" DUPLICATE SEAT NUMBERS ACROSS COUNTERS!");
        System.out.println("Seat #1 was given to Ahmed,Hania, AND Asim!");
    }
}
```

Output from Buggy Code

```
=== MOVIE TICKET BOOKING SYSTEM ===

New TicketCounter created!

Ahmed booked Seat #1

Ahmed booked Seat #2
```

New TicketCounter created!

Hania booked Seat #1

New TicketCounter created!

Asim booked Seat #1

Asim booked Seat #2

--- PROBLEM ---

Ahmed's counter bookings: 2

Hania's counter bookings: 1

Asim's counter bookings: 2

Booked seats: [1, 2]

Booked seats: [1]

Booked seats: [1, 2]

DUPLICATE SEAT NUMBERS ACROSS COUNTERS!

Seat #1 was given to Ahmed,Hania, AND Asim!